

Security Findings Register

SOL DJ Booking Platform — Client-Side Web Application

Report classification	CONFIDENTIAL — contains unredacted system identifiers
Assessment type	Static source code review (manual, white-box, partial scope)
Assessment date	3 September 2026
Target application	SOL DJ Booking Platform (sol.html inline application script, v7)
Backend platform	Google Firebase — Firestore, Authentication, Cloud Storage, Cloud Functions
Firebase project ID	studio-3475382917-e5aaa
API host	https://rork-dj-booking-payment-app.onrender.com
Artefact reviewed	Single client-side JavaScript file (approx. 2,900 lines)
Findings raised	21 security findings; 12 non-security defects
Highest severity	CRITICAL — 5 findings

Handling notice. At the client's instruction this report retains unredacted administrative identifiers, project identifiers and endpoint addresses. In its present form the document constitutes a target list for the application described. It should be transmitted only over encrypted channels, stored only in access-controlled repositories, and should not be forwarded beyond the named recipient without a redacted revision being prepared first.

Prepared as a static review of application source supplied by the client. No dynamic testing, no authenticated exploitation and no interaction with the production environment was performed. Section 2 sets out the limitations that qualify every finding in this register.

1. Scope and Methodology

The client supplied a single client-side JavaScript artefact — the inline application script of `sol.html`, self-identified as version 7 — for review. The application is a two-sided DJ booking marketplace. It authenticates users through Firebase Authentication, reads and writes application state directly to Cloud Firestore from the browser, uploads media to Cloud Storage, invokes two Cloud Functions, and posts booking and payment requests to a REST API hosted on Render.

Review was manual and line-by-line. Each client-originated write to Firestore, each Cloud Function invocation, each REST payload and each HTML rendering path was traced to determine what an authenticated but unprivileged actor could influence. Findings were then classified by the security control that fails rather than by the code construct involved.

2. Limitations of Assessment

These limitations are material. They should be read before any finding in this register is treated as confirmed.

- **Server-side authorisation was not available for review.**

In a Firebase architecture, authorisation is enforced by Firestore Security Rules, Cloud Storage Rules and server-side checks inside Cloud Functions. None of these artefacts were supplied. Findings SOL-01 through SOL-11, SOL-14, SOL-15, SOL-16 and SOL-19 are therefore recorded as **conditional**: each is exploitable if, and only if, the corresponding server-side control is absent or permissive.

- **No dynamic or authenticated testing was performed.**

No account was created, no request was issued to the production environment, and no finding was empirically confirmed. Severity ratings reflect assessed likely impact, not observed impact.

- **Coverage is partial.**

The reviewed artefact references HTML elements, service worker `/sw.js`, Cloud Functions `sendPaypalPayout` and `syncAllAuthUsers`, and REST endpoints under `/api/`. None of these were provided. Additional findings may exist in the unreviewed components.

- **Third-party and payment integrations were not assessed.**

The Stripe checkout flow, PayPal payout mechanism, Mapbox configuration and Firebase project configuration were observed only as referenced from client code.

Recommended next step for the assessing party. Obtaining `firestore.rules`, `storage.rules` and `functions/index.js` would resolve the conditional status of sixteen findings and is the single highest-value action available.

3. Executive Summary

The application places its trust boundary in the wrong place. Authorisation decisions that determine whether a user is an administrator, whether a DJ is verified, whether a booking is owned, whether a payout is due and what a customer is charged are all evaluated in JavaScript running on the user's own device. Code executing in a browser is fully under the control of the person operating that browser; it can be read, modified, or bypassed entirely by issuing requests to Firebase directly. Every such check is therefore an interface convention, not a security control.

Whether this translates into live exposure depends on the server-side rules that were not made available. If those rules are permissive — the common default during development — the assessed impact includes privilege escalation to platform administrator by any registered user, unauthorised initiation of real-money payouts, disclosure of the personal data of every DJ on the platform, and destruction of all transactional records.

Independently of those rules, one finding is exploitable through the application as written. SOL-04 describes stored cross-site scripting in DJ profile fields that renders unescaped in the administrator console. The consequence is that a DJ awaiting approval can seize the administrator's authenticated session at the moment the administrator opens the queue to review them. This converts an unprivileged account into a privileged one without depending on any assumption about server configuration, and it should be treated as the priority item.

3.1 Severity Distribution

Severity	Count	Definition applied in this register
Critical	5	Loss of platform control, funds, or bulk personal data. Remediate immediately.
High	6	Significant unauthorised access, financial loss, or data exposure. Remediate within 7 days.
Medium	8	Meaningful integrity, privacy, or abuse exposure. Remediate within 30 days.
Low	2	Limited impact or requires unusual preconditions. Remediate within 90 days.

CVSS v3.1 base scores are provided where the finding maps cleanly to the framework. Scores assume the conditional precondition in Section 2 is met. Where severity and CVSS diverge, the assigned severity reflects business impact — principally the movement of funds — which the base metrics understate.

4. Findings Register

ID	Finding	Severity	CVSS	Status
SOL-01	Privileged Cloud Functions invocable by any authenticated user	Critical	8.1	Conditional
SOL-02	Privilege escalation to platform administrator via client-writable role flag	Critical	9.9	Conditional
SOL-03	Self-approval as a verified DJ, bypassing the verification workflow	High	8.1	Conditional
SOL-04	Stored cross-site scripting in DJ profile fields, executing in the administrator console	Critical	8.7	Confirmed
SOL-05	SMTP credentials and FCM server key stored in a client-accessible Firestore document	Critical	9.6	Conditional
SOL-06	Booking record tampering and assignment hijack	High	8.1	Conditional
SOL-07	Payout settlement flags stored on client-writable booking documents	High	6.5	Conditional
SOL-08	Transaction pricing calculated client-side and submitted as trusted input	High	6.5	Conditional
SOL-09	Destructive bulk-deletion routine exposed in client code	High	6.5	Conditional
SOL-10	Bulk personal data disclosure through the DJ network directory	High	6.5	Conditional
SOL-11	Unrestricted file upload to a permissively named Cloud Storage path	Medium	6.4	Conditional
SOL-12	Administrative identifiers disclosed in publicly retrievable client source	Medium	5.3	Confirmed
SOL-13	Account enumeration through verbatim authentication error disclosure	Medium	5.3	Confirmed
SOL-14	Forgery of ratings and reviews without transactional ownership verification	Medium	5.4	Conditional
SOL-15	Promotional code collection directly readable by clients	Medium	5.3	Conditional
SOL-16	Sender spoofing and unsolicited messaging in DJ-to-DJ channel	Medium	5.4	Conditional
SOL-17	Persistent user identifiers and personal attributes transmitted to Google Analytics	Medium	—	Confirmed
SOL-18	Precise user geolocation transmitted to unaffiliated third-party services	Medium	5.9	Confirmed
SOL-19	Booking completion driven by client clock, creating payout eligibility	Medium	5.3	Conditional
SOL-20	No electronic mail verification required before transacting	Low	4.3	Confirmed
SOL-21	Formula injection and defective quoting in earnings CSV export	Low	4.3	Confirmed

Status. “Confirmed” denotes a finding evidenced entirely within the supplied artefact. “Conditional” denotes a finding whose exploitability depends on server-side controls that were not supplied for review, per Section 2.

5. Detailed Findings

SOL-01 — CRITICAL

Privileged Cloud Functions invocable by any authenticated user

Affected component	Cloud Functions: sendPaypalPayout, syncAllAuthUsers
Weakness class	CWE-862 Missing Authorization
CVSS v3.1	8.1 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
Verification status	Conditional — depends on server-side controls not supplied

Description

The client invokes two Firebase Callable Functions. `sendPaypalPayout({djId})` initiates a real PayPal transfer to the DJ identified by the supplied parameter. `syncAllAuthUsers({})` enumerates the Firebase Authentication user directory and provisions Firestore user documents. In the client, both are reachable only from the administrator console. However, Callable Functions are addressable by any party holding a valid Firebase ID token for the project; the calling application's user interface is not a constraint on who may invoke them.

Impact

If the function bodies do not independently verify administrative privilege from `context.auth`, any registered user can trigger arbitrary outbound payments and enumerate the full user directory. This finding moves real funds and is ranked first for that reason.

Evidence

```
functions.httpsCallable('sendPaypalPayout');
firebase.functions().httpsCallable('syncAllAuthUsers'). Payout amount is derived client-side by
computeBookingPayoutClient(), whose own comment states it mirrors a server-side equivalent.
```

Remediation

Assert administrative privilege inside each function before any side effect, sourced from a custom auth claim rather than a Firestore document. Recompute the payout amount server-side from booking records. Record payouts in an idempotent ledger within a transaction so a repeated call cannot pay twice.

SOL-02 — CRITICAL

Privilege escalation to platform administrator via client-writable role flag

Affected component	Firestore: users/{uid}.isAdmin
Weakness class	CWE-639 Authorization Bypass Through User-Controlled Key
CVSS v3.1	9.9 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H
Verification status	Conditional — depends on server-side controls not supplied

Description

Administrative status is determined by reading the boolean `isAdmin` from the caller's own user document. The same document is written by the client on every sign-in via `syncUserDoc()`, and the field is written directly by `runAdminUserAction()`. A hardcoded identity check on UID `3i7fQdPjN0Qxz3FysVPvnhtxz1J3` and email `djweirdnasty@gmail.com` provides a secondary path, evaluated in the same client-side context.

Impact

A user who can write their own user document sets the flag and gains the administrator console. That console permits banning and deleting accounts, promoting further administrators, approving DJ verifications, cancelling bookings, initiating payouts, reading the credential store described in SOL-05, and executing the destructive purge described in SOL-09. This is total loss of platform control.

Evidence

`checkAdminStatus()` reads the flag; `runAdminUserAction()` writes `{isAdmin: !isAdmin}`. The founder-protection guard is a client-side string comparison on `btn.textContent`.

Remediation

Move role to a Firebase custom auth claim set exclusively by a privileged Cloud Function. Deny all client writes to role fields in Security Rules. Treat any existing elevated account as suspect and re-verify.

SOL-03 — HIGH

Self-approval as a verified DJ, bypassing the verification workflow

Affected component	Firestore: <code>dj-verifications/{uid}</code> , <code>users/{uid}</code> , <code>dj-status/{uid}</code>
Weakness class	CWE-602 Client-Side Enforcement of Server-Side Security
CVSS v3.1	8.1 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
Verification status	Conditional — depends on server-side controls not supplied

Description

The DJ profile form writes the verification document keyed to the submitting user's own UID, including its `status` field. Whether the value written is 'pending' or 'approved' is decided by the client variable `currentDjVerificationStatus`. The client separately writes `isVerified: true` to its own `dj-status` document and reads `isVerifiedDJ` from its own user document as an alternate approval path.

Impact

Any registered user can mark themselves an approved DJ, appear on the public live map, be surfaced by search, accept bookings and accrue payout entitlement, without ever passing the administrative review the workflow is designed to require. This defeats the platform's trust and safety model, with the reputational and duty-of-care consequences that follow from unvetted individuals being dispatched to private events.

Evidence

```
keepApproved = currentDjVerificationStatus === 'approved', then
db.collection('dj-verifications').doc(user.uid).set(verData). Also
db.collection('dj-status').doc(user.uid).set({isVerified: true}).
```

Remediation

Deny client writes to status and verification fields. Route profile submissions through a Cloud Function that forces pending status. Audit the existing approved set against the administrative review log.

SOL-04 — CRITICAL

Stored cross-site scripting in DJ profile fields, executing in the administrator console

Affected component	Rendering: loadAdminDJs, loadAdminVerifications, renderAdminUsers, renderDJBookings, showDJProfile, createDJMarkerEl
Weakness class	CWE-79 Improper Neutralization of Input During Web Page Generation
CVSS v3.1	8.7 CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N
Verification status	Confirmed — evidenced within the supplied artefact

Description

Attacker-controllable strings are concatenated into HTML and assigned via `innerHTML` throughout the application, with no encoding at any point. Affected values include DJ stage name, biography, avatar URL, client name, event location, special requests, dispute descriptions and review text. Several sites are more exposed still: avatar URLs are interpolated into `onerror` attributes, so the URL need not load for attacker content to execute; and attributes such as `data-client` and `data-dj-name` receive no quote escaping, permitting attribute-context breakout. Only `data-addr` is escaped, and only partially.

Impact

The decisive path is stored, not reflected. A DJ registering an account places a payload in their stage name or avatar field. No privilege is required and no interaction occurs at that time. When an administrator later opens the verification queue or DJ list to review the account, the payload executes in the administrator's authenticated origin, yielding session takeover and, through it, every capability enumerated in SOL-02. The administrator is compromised precisely by performing their duties. Unlike most findings in this register, this one does not depend on Security Rules being permissive.

Evidence

```
card.innerHTML = avatarHtml + '<div><strong>' + djName + ... in loadAdminDJs();
onerror="this.style.display='none'; this.parentElement.innerHTML='" + initial + '"' in
createDJMarkerEl().
```

Remediation

Render user-supplied values with `textContent` or explicit DOM construction. Where markup is unavoidable, encode for the specific context. Replace `data-*` string attributes with closures or the dataset API. Validate that image URLs begin with `https://` and reject inline event handlers. Deploy a Content-Security-Policy prohibiting inline script as defence in depth.

SOL-05 — CRITICAL**SMTP credentials and FCM server key stored in a client-accessible Firestore document**

Affected component	Firestore: config/notification-settings
Weakness class	CWE-522 Insufficiently Protected Credentials
CVSS v3.1	9.6 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N
Verification status	Conditional — depends on server-side controls not supplied

Description

The administrative settings panel reads and writes a single Firestore document holding the outbound mail server hostname, port, username and password in cleartext, together with the Firebase Cloud Messaging server key and project identifier. Both the read on panel load and the write on save are performed directly by the browser.

Impact

Disclosure yields the ability to send electronic mail authenticated as the platform — enabling convincing phishing of the platform's own users — and to dispatch push notifications to the entire installed base. Because these credentials govern systems outside the application, impact extends beyond the vulnerable component. Credential reuse, if present, extends it further still.

Evidence

`loadAdminSettings()` reads `config/notification-settings`, populating `sol-settings-smtp-pass` and `sol-settings-fcm-key`.

Remediation

Treat both credentials as disclosed and rotate them now, irrespective of the Rules outcome. Relocate to Cloud Functions environment configuration or Secret Manager. Delete the Firestore document. Deny all client access to the `config` collection. Review mail and push logs for unauthorised use.

SOL-06 — HIGH**Booking record tampering and assignment hijack**

Affected component	Firestore: bookings/{id}
Weakness class	CWE-639 Authorization Bypass Through User-Controlled Key
CVSS v3.1	8.1 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
Verification status	Conditional — depends on server-side controls not supplied

Description

Booking state transitions are performed as direct client writes. Accepting a request writes `{status: 'confirmed', djId: user.uid}` to the booking document — the client asserts its own ownership of the record as part of the same operation that transitions it. Cancellation, arrival marking, status history and refund determination follow the same pattern.

Impact

An actor able to enumerate the bookings collection can assign themselves as the assigned DJ on bookings belonging to others, thereby diverting the associated payout; cancel arbitrary bookings; or mark events completed to accelerate payout eligibility. Booking documents also carry client name, electronic mail address, telephone number and event address, so read access alone constitutes a personal data exposure.

Evidence

```
var updateData = {status:newStatus, updatedAt:..., djId:user.uid} in handleBookingAction().
```

Remediation

Restrict booking reads to the associated client and assigned DJ. Deny client writes to status, djId, refund and payout fields. Perform transitions in a Cloud Function that verifies the caller's relationship to the booking and enforces a legal state machine.

SOL-07 — HIGH

Payout settlement flags stored on client-writable booking documents

Affected component	Firestore: bookings/{id}.payoutSent, depositPayoutSent, finalPayoutSent
Weakness class	CWE-840 Business Logic Errors
CVSS v3.1	6.5 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N
Verification status	Conditional — depends on server-side controls not supplied

Description

Whether a DJ has been paid is recorded as boolean fields on the booking document.

`computeBookingPayoutClient()` reads these to decide the outstanding amount displayed to the administrator and offered to the payout function.

Impact

If a DJ can write their own booking documents, clearing a settlement flag restores the booking to an unpaid state and makes it eligible for payment a second time. Combined with SOL-01, settlement can be re-triggered without administrative involvement. The result is direct, repeatable financial loss that reconciliation may not surface promptly.

Evidence

```
var depositPaid = legacyPaid || !!b.depositPayoutSent;
```

Remediation

Record settlement in a separate ledger collection writable only by the payout function. Make payout operations idempotent on a server-generated key. Reconcile against the PayPal transaction record rather than application state.

SOL-08 — HIGH**Transaction pricing calculated client-side and submitted as trusted input**

Affected component	REST: POST /api/bookings/request, POST /api/payments/create-checkout-session
Weakness class	CWE-602 Client-Side Enforcement of Server-Side Security
CVSS v3.1	6.5 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N
Verification status	Conditional — depends on server-side controls not supplied

Description

The booking payload transmits `promo_discount`, `duration`, `dj_count`, `deposit_only`, `equipment`, `agreed_terms`, `agreed_waiver` and `signature`, all computed or collected in the browser. The promotional discount in particular is calculated locally after a direct read of the `promo-codes` collection.

Impact

If the API derives the Stripe charge from any submitted value, an actor may set the discount to the full amount and obtain services at no cost. The contractual acknowledgement fields are equally forgeable, which weakens the evidentiary basis of the terms and liability waiver the platform relies upon.

Evidence

`promo_discount`: `promoDiscount` within the payload object of the `sol-quick-form` submit handler.

Remediation

Recompute the total server-side from the persisted booking, the authoritative rate card and a server-side promotional code lookup. Treat the submitted total as advisory display data only. Record acknowledgement server-side at time of receipt.

SOL-09 — HIGH**Destructive bulk-deletion routine exposed in client code**

Affected component	Client function: <code>purgeTestData</code> / <code>purgeCollection</code>
Weakness class	CWE-732 Incorrect Permission Assignment for Critical Resource
CVSS v3.1	6.5 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H
Verification status	Conditional — depends on server-side controls not supplied

Description

A routine reachable from the administrator console iterates and deletes the entire contents of the bookings, tips, disputes, feedback, setlists, playlists, saved-djs and conversations collections, then resets every `dj-status` document. The only barriers are a client-side `isAdmin` check and a browser confirmation dialogue.

Impact

Complete and unrecoverable loss of all transactional and communications history, absent point-in-time backup. This is destructive rather than merely disclosing, and is reachable by anyone who satisfies SOL-02 or SOL-04. The routine is labelled as operating on test data but is not scoped to test records in any way.

Evidence

```
function purgeTestData() guarded by if (!isAdmin) { alert('Admin only.');
```

Remediation

Remove the routine from client code entirely. If bulk deletion is operationally necessary, implement it as an out-of-band administrative script requiring separate credentials. Confirm Firestore point-in-time recovery is enabled.

SOL-10 — HIGH

Bulk personal data disclosure through the DJ network directory

Affected component	Firestore: dj-verifications collection query
Weakness class	CWE-200 Exposure of Sensitive Information to an Unauthorized Actor
CVSS v3.1	6.5 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Verification status	Conditional — depends on server-side controls not supplied

Description

`loadDJNetwork()` retrieves every document in `dj-verifications` with approved status in order to render a list of names. Those documents embed the full profile: telephone number, PayPal account identifier, electronic mail address, and city and state of residence.

Impact

Any user with read access — which, per SOL-03, may include self-approved accounts — retrieves the complete contact and payment-identifier set for every DJ on the platform. PayPal identifiers alongside names and telephone numbers form a directly actionable target set for payment-redirection fraud against the DJ population. Depending on the residency of affected individuals this may constitute a notifiable personal data breach.

Evidence

```
db.collection('dj-verifications').where('status','==','approved').get(), rendering d.email || p.email.
```

Remediation

Maintain a separate public projection containing only display name, avatar, genres and rating. Restrict the verification collection to the owning user and administrators. Assess notification obligations with counsel.

SOL-11 — MEDIUM**Unrestricted file upload to a permissively named Cloud Storage path**

Affected component	Cloud Storage: public/{uid}/dj-avatars/, public/{uid}/dj-galleries/
Weakness class	CWE-434 Unrestricted Upload of File with Dangerous Type
CVSS v3.1	6.4 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:L/A:L
Verification status	Conditional — depends on server-side controls not supplied

Description

Uploads are written to paths under a public/ prefix. Content type is checked only by `file.type.match('image.*')` in the browser, a value the client supplies and can therefore set arbitrarily. No size limit is imposed at any layer visible in the artefact, and the stored extension is taken from the user-supplied filename.

Impact

If Storage Rules mirror the path naming, arbitrary content may be stored and served from infrastructure associated with the platform, supporting malware distribution or phishing content that inherits the platform's apparent legitimacy. The absence of a size limit additionally permits storage and egress cost amplification.

Evidence

```
var path = 'public/' + user.uid + '/dj-avatars/' + Date.now() + '.' + ext;
```

Remediation

Enforce content type and a size ceiling in Storage Rules rather than in the browser. Restrict writes to the authenticated user's own prefix. Normalise extensions server-side. Consider server-side image re-encoding to strip embedded content.

SOL-12 — MEDIUM**Administrative identifiers disclosed in publicly retrievable client source**

Affected component	Constants: ADMIN_UID, ADMIN_EMAIL
Weakness class	CWE-200 Exposure of Sensitive Information to an Unauthorized Actor
CVSS v3.1	5.3 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Verification status	Confirmed — evidenced within the supplied artefact

Description

The application embeds `ADMIN_UID = '3i7fQdPjN0Qxz3FysVPvnhtxz1J3'` and `ADMIN_EMAIL = 'djweirdnasty@gmail.com'` as literals. Any visitor may retrieve these by viewing source.

Impact

These values do not permit impersonation directly, as Firebase enforces address uniqueness within a project. They do identify, without ambiguity, the single account whose compromise yields platform control — converting an untargeted attack into a targeted one against a named individual. Credential stuffing, password reset interception and social engineering all become materially more tractable. No multi-factor requirement is evident in the reviewed artefact.

Evidence

```
const ADMIN_UID = '3i7fQdPjN0Qxz3FysVPvnhtxzLJ3';
```

Remediation

Enable multi-factor authentication on the identified account as an immediate measure. Remove the constants and derive privilege from custom claims per SOL-02.

SOL-13 — MEDIUM

Account enumeration through verbatim authentication error disclosure

Affected component	Authentication handlers: sign-in, sign-up, password reset
Weakness class	CWE-204 Observable Response Discrepancy
CVSS v3.1	5.3 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Verification status	Confirmed — evidenced within the supplied artefact

Description

Firebase authentication errors are surfaced to the interface unmodified via `authStatus.textContent = err.message`. Firebase distinguishes a non-existent account from an incorrect password, and reports the distinction in the message text.

Impact

An unauthenticated party can determine whether a given electronic mail address holds an account, producing a validated address list for phishing and credential stuffing. The password reset handler exposes the same distinction and is subject to no rate limiting within the reviewed artefact.

Evidence

```
.catch(function(err) { authStatus.textContent = err.message; })
```

Remediation

Present a single generic failure message for all authentication outcomes. Retain specific codes in logs only. Enable Firebase email enumeration protection and apply App Check.

SOL-14 — MEDIUM**Forgery of ratings and reviews without transactional ownership verification**

Affected component	Firestore: feedback collection
Weakness class	CWE-345 Insufficient Verification of Data Authenticity
CVSS v3.1	5.4 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:N
Verification status	Conditional — depends on server-side controls not supplied

Description

Review submission writes a feedback document specifying `toUserId`, `rating` and `bookingId`. The recipient and booking identifiers are read from DOM attributes on the originating control, and no server-side check establishes that the submitting user held the referenced booking.

Impact

Ratings, which drive search ordering, the computed match score and consumer trust, can be inflated for a favoured DJ or suppressed for a competitor at arbitrary volume. Review text is additionally rendered through the unescaped path in SOL-04, making this a delivery vector for that finding.

Evidence

```
rateDjId = btn.getAttribute('data-rate-dj'), written unchecked to feedback.
```

Remediation

Accept reviews only through a Cloud Function that confirms the submitting user was the client on a completed booking involving the named DJ, and that no review already exists for it.

SOL-15 — MEDIUM**Promotional code collection directly readable by clients**

Affected component	Firestore: promo-codes collection
Weakness class	CWE-200 Exposure of Sensitive Information to an Unauthorized Actor
CVSS v3.1	5.3 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N
Verification status	Conditional — depends on server-side controls not supplied

Description

Code validation is performed by a direct client read of `promo-codes/{CODE}`. Read access sufficient for validation is generally sufficient for enumeration of the collection.

Impact

All promotional codes, discount values and active flags become discoverable, including codes intended for restricted distribution or not yet launched. Discounts may be applied outside their intended campaign, audience or period. Compounded by SOL-08, under which the resulting discount is submitted as trusted input.

Evidence

```
db.collection('promo-codes').doc(code).get()
```

Remediation

Validate and apply codes exclusively server-side. Deny client read access to the collection. Return only a boolean validity result and the resulting price.

SOL-16 — MEDIUM

Sender spoofing and unsolicited messaging in DJ-to-DJ channel

Affected component	Firestore: dj-messages collection
Weakness class	CWE-345 Insufficient Verification of Data Authenticity
CVSS v3.1	5.4 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:N
Verification status	Conditional — depends on server-side controls not supplied

Description

Messages are written client-side with caller-supplied `toUid`, `toName` and `fromName`. Only `fromUid` derives from the authenticated session, and `display` consumes the free-text name field.

Impact

Messages may be addressed to any user and attributed to any display name, enabling impersonation of platform staff or other DJs for social engineering. There is no rate limit, permitting bulk unsolicited messaging. Message bodies are collected via `prompt()` and rendered without encoding.

Evidence

```
db.collection('dj-messages').add({fromName: user.displayName || user.email, toUid: toUid, ...})
```

Remediation

Derive sender identity server-side from the auth context and never from a submitted field. Restrict recipients to permitted relationships. Apply write rate limiting.

SOL-17 — MEDIUM

Persistent user identifiers and personal attributes transmitted to Google Analytics

Affected component	Analytics: trackSolEvent
Weakness class	CWE-359 Exposure of Private Personal Information to an Unauthorized Actor
CVSS v3.1	— Privacy and compliance finding; CVSS base metrics not applicable
Verification status	Confirmed — evidenced within the supplied artefact

Description

Analytics events transmit the Firebase user identifier as a `uid` parameter to Google Analytics, alongside stage name, city, hourly rate and booking identifiers, across sign-up, sign-in, registration, verification, booking and review events.

Impact

Transmitting persistent user identifiers to Google Analytics contravenes the Google Analytics terms of service, which prohibit sending data that permits identification of an individual, and creates exposure under GDPR, CCPA and comparable regimes where the identifier is linkable to a natural person. Consequences include property suspension and regulatory liability. No consent mechanism governing this transmission is evident.

Evidence

```
trackSolEvent('sign_up', { method:'email', uid: cred.user.uid });
trackSolEvent('dj_registration', { uid, stage_name, city, hourly_rate })
```

Remediation

Remove identifiers and personal attributes from analytics payloads. Substitute a salted pseudonymous identifier if cohort analysis is required. Review retained historical data and implement consent management.

SOL-18 — MEDIUM

Precise user geolocation transmitted to unaffiliated third-party services

Affected component	External: nominatim.openstreetmap.org, router.project-osrm.org
Weakness class	CWE-359 Exposure of Private Personal Information to an Unauthorized Actor
CVSS v3.1	5.9 CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N
Verification status	Confirmed — evidenced within the supplied artefact

Description

Reverse geocoding, forward geocoding, address autocomplete and route computation are performed by direct browser requests to the OpenStreetMap Nominatim and OSRM public demonstration endpoints. Requests carry precise coordinates, including the live coordinates of DJs sharing location while travelling to engagements.

Impact

Coordinates of identifiable users, together with the requesting address, transit to operators with whom the platform holds no data processing agreement. Both endpoints are published for evaluation rather than production use and impose usage policies the application does not observe; availability is not guaranteed and blocking is a realistic operational risk. Location data of this precision is treated as sensitive under several privacy regimes.

Evidence

```
fetch('https://nominatim.openstreetmap.org/reverse?format=json&lat=' + lat + '&lon=' + lng
...)
```

Remediation

Proxy geocoding and routing through the platform's own backend under a commercial agreement with defined processing terms. Disclose third-party processing in the privacy notice.

SOL-19 — MEDIUM**Booking completion driven by client clock, creating payout eligibility**

Affected component	Client function: renderDJBookings automatic completion loop
Weakness class	CWE-807 Reliance on Untrusted Inputs in a Security Decision
CVSS v3.1	5.3 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N
Verification status	Conditional — depends on server-side controls not supplied

Description

On each snapshot the DJ console iterates confirmed bookings, compares the event date against the browser's clock and writes `{status: 'completed'}` to any that appear past. Completion is the condition governing final payout eligibility in `computeBookingPayoutClient()`.

Impact

The transition that unlocks payment is triggered by an unauthenticated, user-controllable clock. Advancing the system clock marks future engagements complete and accrues payout entitlement for services not rendered. Because every open console performs the write, concurrent writes to the same document are also likely.

Evidence

```
if (d < now) { db.collection('bookings').doc(b.id).set({status: 'completed', ...}) }
```

Remediation

Perform completion in a scheduled Cloud Function using server time. Deny client writes to the status field. Require confirmation from both parties, or a defined settlement window, before payout eligibility.

SOL-20 — LOW**No electronic mail verification required before transacting**

Affected component	Authentication: onAuthStateChanged gate
Weakness class	CWE-287 Improper Authentication
CVSS v3.1	4.3 CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N
Verification status	Confirmed — evidenced within the supplied artefact

Description

The application gate checks only that a user object exists. `emailVerified` is never consulted, so an account may book, message, review and submit DJ verification with an unverified and potentially unowned address.

Impact

Supports disposable and impersonated account creation, degrades the reliability of booking correspondence, and reduces the evidentiary value of the electronic mail address recorded against contractual acknowledgements.

Evidence

```
auth.onAuthStateChanged(function(user) { if (user) { solGate.style.display = 'none'; ... } })
```

Remediation

Require verification before booking or DJ submission. Mirror the requirement in Security Rules using the token's verification claim so it cannot be bypassed client-side.

SOL-21 — LOW

Formula injection and defective quoting in earnings CSV export

Affected component	Client function: loadDJEarnings CSV export handler
Weakness class	CWE-1236 Improper Neutralization of Formula Elements in a CSV File
CVSS v3.1	4.3 CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
Verification status	Confirmed — evidenced within the supplied artefact

Description

The export wraps each cell in double quotes by concatenation, without doubling quotes occurring within the value and without neutralising leading =, +, - or @ characters. Cell values include client name and event type, both attacker-controllable.

Impact

A value beginning with a formula character is interpreted as a formula on opening in common spreadsheet applications, permitting command execution on the machine of the DJ or accountant opening the export. A value containing a double quote corrupts the structure of the file, misaligning financial columns.

Evidence

```
rows.map(function(r) { return r.map(function(c) { return '"' + c + '"'; }).join(','); })
```

Remediation

Double embedded quotes per RFC 4180 and prefix any value beginning with a formula character with an apostrophe or single leading quote.

6. Appendix A — Non-Security Defects

The following defects were identified during review. They are recorded for completeness and are not assigned severity ratings, as none has a direct security consequence. Two carry indirect financial or contractual exposure and are marked accordingly.

Ref	Defect	Description
A-01	Unbounded timer accumulation	The countdown routine re-schedules itself with <code>setTimeout</code> and is never cancelled. As the enclosing render runs on every Firestore snapshot, each render creates additional permanent timers that continue updating detached DOM nodes. Performance degrades monotonically for the duration of the session.
A-02	Unreachable booking progress states	In the client booking progress indicator, the branch <code>else if (status === 'confirmed' && djSharing)</code> is preceded by an unqualified <code>status === 'confirmed'</code> branch and can never execute; the <code>arrived</code> and <code>completed</code> branches are likewise unreachable. Steps three through five never display. An abandoned computation of <code>dots</code> , <code>connectors</code> and <code>dotsRow</code> above the loop produces no output.
A-03	Date parsing timezone defect — contractual exposure	<code>new Date('YYYY-MM-DD')</code> parses as UTC midnight, resolving to the preceding evening in all Western Hemisphere timezones. This shifts the countdown, the availability calendar, the automatic completion comparison and, materially, the seven-day cancellation refund boundary — which may therefore be computed a day adrift of the published policy.
A-04	Duplicated cancellation handlers	Cancellation controls receive a listener within the render function and a second through a delegated document-level listener, producing two confirmation dialogues and two writes per activation.
A-05	Batch write limit exceeded in merge routine	<code>mergedjs()</code> accumulates bookings, tips, disputes, feedback and saved-DJ updates into a single <code>writeBatch</code> . Firestore caps batches at 500 operations; the routine will throw for any DJ exceeding that volume, having already deleted source documents in the same batch.
A-06	Duplicated DOM identifiers	The identifier <code>sol-song-list</code> is consumed by both the client playlist and the song suggestion modal; <code>sol-dj-upcoming-events</code> is consumed by both the DJ console and the DJ profile modal. Each pair overwrites the other.
A-07	Notification emission on initial snapshot	<code>subscribeDJBookings()</code> treats the initial snapshot's added events as new arrivals, emitting a desktop notification for every pre-existing pending booking each time the console is opened.
A-08	Refund amount recorded as a literal	Cancellation writes <code>refundAmount: 50</code> , a fixed integer, where the interface represents the entitlement as fifty per cent of the booking value. Recorded refund obligations will be incorrect for any booking other than one hundred units.
A-09	Undefined values in analytics payload	The settings save event transmits <code>data.pushEnabled</code> and <code>data.emailEnabled</code> ; the constructed object exposes these as <code>data.push.enabled</code> and <code>data.email.enabled</code> . Both parameters are undefined.
A-10	Silent failure on missing composite index	<code>loadPlaylist()</code> combines an equality filter with an ordering on a different field, requiring a composite index. Absent the index the query rejects into a catch block that clears the container, presenting an empty playlist rather than an error.

Ref	Defect	Description
A-11	Listener accumulation on carousel and fallback subscription	<code>renderDjCarousel()</code> attaches a scroll listener to the same element on every render. The fallback <code>onSnapshot</code> in <code>subscribeToDJs()</code> is never unsubscribed. The control <code>sol-refresh-djs</code> carries two competing click handlers.
A-12	Identity determined by substring match	<code>showDJProfile()</code> attributes a hardcoded event schedule to any profile whose name contains “weird” or “nasty”. Unrelated DJs will display another party's engagements, with the misrepresentation risk that implies.

7. Appendix B — Remediation Sequence

Findings are grouped below by the order in which addressing them reduces the most exposure per unit of effort. The first group contains actions whose effect does not depend on resolving the conditional status in Section 2 and which should proceed regardless.

Immediate — proceed without waiting for further review

Action	Refs	Note
Rotate the SMTP password and FCM server key	SOL-05	Treat as disclosed. Rotation is independent of whether Rules permitted access.
Enable multi-factor authentication on the administrator account	SOL-12	The account is named in public source. Minutes of effort.
Confirm Firestore point-in-time recovery is enabled	SOL-09	Establishes a recovery position before any other work begins.
Retrieve firestore.rules, storage.rules and functions/index.js	—	Resolves the conditional status of sixteen findings.

Week one — close the confirmed path and the money paths

Action	Refs	Note
Encode all user-supplied values on output	SOL-04	The one finding not contingent on server configuration. Convert innerHTML rendering to textContent or explicit DOM construction and add a Content-Security-Policy.
Add administrative assertions inside both Cloud Functions	SOL-01	Verify privilege from context.auth before any side effect; recompute payout server-side.
Publish Security Rules denying client writes to privileged fields	SOL-02, 03, 06, 07, 19	Deny writes to isAdmin, isVerifiedDJ, verification status, booking status, djId and payout flags. Single highest-leverage change in the register.
Recompute transaction totals server-side	SOL-08	Treat all client-submitted pricing as display data.
Remove the bulk purge routine from client code	SOL-09	Deletion, not disclosure. Removal is faster than securing it.

Month one — data exposure and abuse resistance

Action	Refs	Note
Introduce a public DJ projection collection	SOL-10	Removes telephone, PayPal identifier and address from client-reachable documents.
Constrain Storage Rules by type, size and path	SOL-11	
Move role assignment to custom auth claims	SOL-02	

Action	Refs	Note
Generalise authentication error messaging; enable App Check	SOL-13	
Validate reviews, messages and promotional codes server-side	SOL-14, 15, 16	
Remove identifiers from analytics; proxy geocoding and routing	SOL-17, 18	Compliance exposure; consider reviewing with counsel.
Require verified electronic mail before transacting	SOL-20	

Backlog — correctness and hygiene

Action	Refs	Note
Correct CSV export quoting and formula neutralisation	SOL-21	
Address Appendix A defects	A-01 to A-12	Prioritise A-03 and A-08, which affect refund calculation and recorded refund obligations.

End of Register

This report reflects a static review of a single client-side artefact as supplied on 3 September 2026. It does not constitute a certification of security, and absence of a finding is not evidence of absence of a vulnerability. The limitations in Section 2 qualify every statement made in this document. Findings marked Conditional require confirmation against the server-side controls named therein before they are recorded as resolved or dismissed.